C++ Memory Management: Virtual Destructors & Multiple Inheritance

Virtual Destructors: The Basics

Consider the following standard class inheritance setup in C++:

```
class Base {
public:
    ~Base() { }
};

class Child : public Base {
public:
    ~Child() { }
};
```

Now in `main` if I write `Child obj;`, then the child destructor is called and after, the Base destructor is called.

Case 2: The Problem with Non-Virtual Destructors

However, what happens if we instantiate the child dynamically and hold it via a base pointer?

```
Base * ptr = new Child();
delete ptr;
```

Without the `virtual` keyword, C++ resolves function calls based on the pointer's **static type** and not the actual object it points to. So since the compiler saw only `Base *`, it generates code to run `~Base()`.

It does not perform a dynamic lookup in the vtable, so `~Child()` is completely ignored. Its memory owned by the child class is never released.

This is classified as undefined behaviour because `delete` needs to know the size of the memory block being freed. The non-virtual destructor passes `sizeof(Base)` and not `sizeof(Child)`. This can corrupt the heap.

DELETING A CHILD OBJECT THROUGH A BASE CLASS POINTER WHEN THE BASE DESTRUCTOR IS NON-VIRTUAL RESULTS IN UNDEFINED BEHAVIOUR.

2 Solutions to this problem:

1. Declare a virtual destructor, i.e., `virtual ~Base()`
2. Use a `protected` destructor if a base class is not meant to be deleted through a base pointer.

How Virtual Destructor Helps

Instead of trusting the pointer type, the program follows the object's hidden `vptr` to its `vtable` to find the actual destructor for the object's runtime type, which is `Child`. Runtime calls `~Derived()` and then calls `~Base()`.

Multiple Inheritance & Pointer Offset Mismatch

In case of multiple inheritance (a child inheriting from one or more parent classes), there is a Pointer Offset Mismatch.

```
class Base1 {
public:
    int a = 10;
};

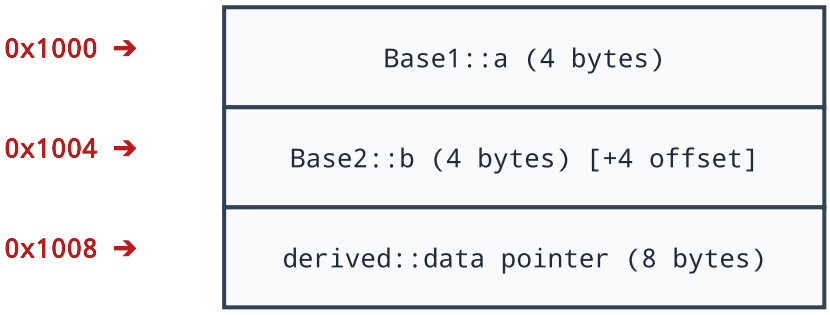
class Base2 {
public:
    int b = 20;
};

class Derived : public Base1, public Base2 {
public:
    int* data = new int[100]; // dynamically allocated
    ~Derived() { delete [] data; }
};
```

Memory Layout: Without Virtual Destructor (Case 1)

A derived object allocated at memory address `0x1000` will look like this:

1. 4 bytes for `Base1::a`
2. 4 bytes for `Base2::b` (+4 offset)
3. 8 bytes for `derived::data` pointer



Now if we write:

```
Derived * d = new Derived();
Base2 * b2 = d;
delete b2;
```

`d` holds `0x1000`. The compiler knows `Base2` starts at 4 bytes after `Base1`. To make `b2` point to `Base2`, the compiler adds `+4` to the address, so `b2 = 0x1004`.

Without virtual, the compiler relies on the static type `Base2*` pointer. It does 2 wrong things:

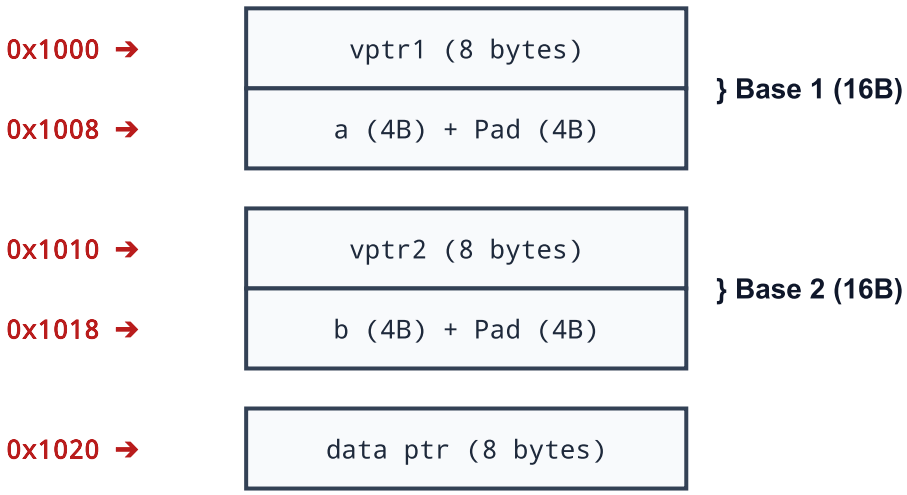
The Compiler Does 2 Wrong Things Here:

1. It calls `~Base2()` and skips `~Derived()`, so (100 x 4) 400 bytes of the dynamically allocated array is leaked.
2. It calls `delete(0x1004, sizeof(Base2) -> 4)`. So instead of deleting 4+4+8 = 16 bytes of data starting from `0x1000`, it deletes just a middle chunk, destroying heap integrity.

Memory Layout: With Virtual Destructor (Case 2)

If we add `virtual ~Base1() = default;` and `virtual ~Base2() = default;`, this forces the compiler to insert a hidden 8 byte `vptr` into each base subobject. So now:

- **[Base 1]**
    1. 8 bytes of vptr pointing to primary Vtable.
    2. 4 bytes `a`, 4 byte padding (TOTAL 16 bytes).
- **[Base 2]**
    1. 8 bytes vptr pointing to secondary Vtable.
    2. 4 bytes `b`, 4 byte padding (TOTAL 16 bytes).



The Role of the 'Thunk'

Now when `Base2* b2 = d;` executes, `d` holds `0x1000`, and the compiler shifts the `b2` pointer by `+16` bytes to the Base 2 sub-object.

Now `delete b2` is executed. Since `base2` is virtual, the CPU reads the `vptr` stored at the start of the Base 2 subobject which leads to the secondary vtable. Now the destructor slot in this secondary vtable points to a helper function called a **thunk**.

The thunk knows `Base2` sits at `+16` inside derived, it subtracts 16 from `b2` restoring the true address of derived. Now `~Derived()` runs first, followed safely by `delete(0x1000, sizeof(derived));`.



Golden Rule:
Whenever we delete derived objects through base pointers, or use base class abstract base classes ➔ **we should always use a virtual destructor.**